

DSA DAY-1 TOPICS COVERS

Arrays

Array: A collection of elements stored in contiguous memory locations.

Indexing: Elements are accessed using an index, starting from 0.

Access Time: $O(1)$ (constant time).

Insertion/Deletion: $O(n)$ (elements may need to be shifted).

Time Complexity: Measures how the execution time grows as the input size increases.

Helps compare the efficiency of different algorithms.

Common complexities: $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$.

Space Complexity: Measures how much extra memory an algorithm uses.

Lower space complexity means better memory efficiency.

Don't Memorize

Focus on understanding the logic behind concepts instead of memorizing formulas.

Practice coding to build intuition.

Why do arrays start from index 0?

Because the first element is stored at the base memory address, so its offset is 0.

Why is $O(n)$ better than $O(n^2)$?

$O(n)$ grows linearly with input size, while $O(n^2)$ grows much faster, making it slower for large datasets.

Why do interviewers care about complexity?

It shows whether you can write efficient, scalable, and optimized code that performs well with large inputs.

What is Brute Force?

Brute Force means solving a problem in the most simple and straightforward way, without trying to optimize it.

Why is it called brute force?

Because it tries every possible combination without using any smart trick or optimization.

Time Complexity

Brute Force: $O(n^2)$ (two nested loops)

Optimized (using Hash Map): $O(n)$

Remember for interviews

Brute Force = Simple but Slow

Optimized = Smart and Fast

Day 1 - Arrays (LeetCode)

1. Two Sum

Problem

The screenshot displays the LeetCode interface for the 'Two Sum' problem. On the left, the problem description is visible, including the goal: 'Given an array of integers nums and an integer target, return indices of the two numbers such that they add up to target.' It also provides three examples with their inputs, outputs, and explanations. The middle section shows the submission status as 'Accepted' with 63/63 testcases passed, a runtime of 43 ms (beating 24.11%), and a memory usage of 14.08 MB (beating 81.74%). A runtime distribution graph is also shown. On the right, the C++ code is displayed, which uses a nested loop to find the two numbers that sum to the target.

```
1 class Solution {
2 public:
3     vector<int> twoSum(vector<int>& nums, int target) {
4         for(int i=0; i<nums.size(); i++){
5             for(int j=i+1; j<nums.size(); j++){
6                 if(nums[i]+nums[j]==target)
7                     return {i,j};
8             }
9         }
10    }
11    return {};
12 };
```

Given an array `nums` and a target value, return the indices of the two numbers whose sum equals the target.

Brute Force Approach

Use two nested loops.

Check every possible pair.

If `nums[i] + nums[j] == target`, return the indices.

Time Complexity

$O(n^2)$

Space Complexity

$O(1)$

Key Learning

Learned nested loops.

Learned brute-force approach.

Learned how to return an array in LeetCode using `malloc` (C) / `vector` (C++).

Understood why checking every pair is slower than an optimized solution.

2. Contains Duplicate

Problem

The screenshot shows the LeetCode interface for problem 217, "Contains Duplicate". The problem description states: "Given an integer array `nums`, return `true` if any value appears at least twice in the array, and return `false` if every element is distinct." Examples provided include `[1,2,3,1]` returning `true` and `[1,2,3,4]` returning `false`. The user's submission, written in C++, is shown on the right, featuring a nested loop approach. A red banner at the top of the code editor indicates "Time Limit Exceeded" for 70/79 test cases. The left sidebar shows the problem is "Attempted" and has a "14.1K" upvote count.

217. Contains Duplicate Attempted

Easy Topics Companies

Given an integer array `nums`, return `true` if any value appears at least twice in the array, and return `false` if every element is distinct.

Example 1:

Input: `nums = [1,2,3,1]`

Output: `true`

Explanation: The element 1 occurs at the indices 0 and 3.

Example 2:

Input: `nums = [1,2,3,4]`

Output: `false`

Explanation: All elements are distinct.

Example 3:

14.1K 454

Time Limit Exceeded 70 / 79 testcases passed

Last Executed Input

`nums =`

`[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 89]`

View more

Code C++

```
1 class Solution {
2 public:
3     bool containsDuplicate(vector<int>& nums) {
4         for(int i=0; i<nums.size(); i++){
5             for(int j=i+1; j<nums.size(); j++){
6                 if(nums[i]==nums[j]){
7                     return true;
8                 }
9             }
10        }
11        return false;
12    }
13 }
14 }
```

Saved Ln 6, Col 33

Testcase Test Result

Given an integer array, return true if any element appears more than once; otherwise, return false.

Brute Force Approach

Compare every element with all remaining elements.

If any two elements are equal, return true.

If no duplicates are found after checking all pairs, return false.

Time Complexity

$O(n^2)$

Space Complexity

$O(1)$

Key Learning

Learned how to compare all pairs using nested loops.

Understood that return false should be outside the loops.

Learned the brute-force pattern for searching.